

Executable JavaScript as a Specification Language: A JS-SAM Case Study on SysMoBench

Jean-Jacques Dubray
jdubray@gmail.com

In collaboration with the SysMoBench / Specula team

July 2, 2026 (draft)

Abstract

SysMoBench evaluates whether large language models (LLMs) can formally model real-world concurrent and distributed systems, grading generated TLA^+ specifications through four automated phases: syntax, bounded model checking, conformance against execution traces captured from the real system, and invariant verification. We extend SysMoBench with its first *non-formal-language* backend, JS-SAM: specifications written in JavaScript following the SAM pattern, whose semantics — actions propose, a model accepts, state is a pure function of the model — are deliberately aligned with TLA^+ . The backend tests a specific hypothesis: LLMs may model systems more accurately in a language ubiquitous in their training data than in formal languages they rarely see.

Using the Asterinas OS spinlock task, we instrument the real kernel, run it under QEMU, and capture a 28-window transition corpus; we then evaluate four Claude models (Haiku 4.5, Sonnet 4.6, Opus 4.8, Fable 5) end to end. Four findings emerge. First, trace conformance is the only phase with discriminating power: all four models pass syntax, model checking, and invariant verification, while conformance spreads them from 21.4% to 89.3% — including a frontier model that silently no-ops an entire action (every `ReleaseLock`) yet passes every internal-consistency phase. Second, a single round of repair-from-trace-feedback separates the models into three clean tiers (full, partial, and no self-correction), and shows that one-shot accuracy and repair ability are different axes: the model with the categorical release defect repairs to 100%, while a mid-tier model given identical feedback cannot. Third, on the same task and models, 4/4 JS-SAM specifications are end-to-end checkable versus 1/4 for TLA^+ , attributable to a concrete TLA^+ semantic pitfall (unbounded `CHOOSE`) that JavaScript’s native `null` sidesteps, and to JS-SAM’s self-contained executable artifact eliminating the model-checker configuration surface. Finally, a pre-registered, paired, four-arm study ($N=5$ generations per cell) puts the conformance question itself under control. Most of an apparent TLA^+ conformance advantage was *prompt prescriptiveness*, not language: adding the identical semantics block to the JavaScript prompt closed most of the gap. A one-directional residual survives that control — TLA^+ reaches 100% conformance for every model and every generation, constrained JS-SAM does so for one model of four — but a plain-JavaScript control arm locates its cause: a bare `next(state, action, data)` transition function, structurally isomorphic to the TLA^+ relation, ties TLA^+ at 100% for every model and every generation. The residual is therefore attributable neither to JavaScript nor to formality but to the SAM specification contract’s defect surface (proposal/acceptor wiring plus mandatory auxiliary state); what models transcribe reliably is *minimal declarative shape*, in any language. The hypothesis splits: familiarity and a self-contained artifact buy checkability; transcription fidelity is governed by the shape of the specification contract — a directly actionable design result for JS-SAM. We

report these results candidly as a single-task case study, describe the kernel trace-capture and controlled-comparison methodology in full, and outline the path to a multi-task comparison.

1 Introduction

Formal specifications are among the most effective tools for establishing the correctness of concurrent and distributed systems [7], and among the least used: writing them demands rare expertise and sustained effort. LLMs raise an obvious question — can they write specifications for us? — and a harder one: *how would we know if they got it right?* A generated specification can be syntactically flawless, explore cleanly under a model checker, satisfy plausible invariants, and still describe a system that does not exist. The SysMoBench benchmark [1] confronts this directly. Its authors observed that when asked to model Etcd’s Raft implementation, a frontier LLM produced, in effect, the specification from the appendix of the Raft paper — a “textbook” model with little connection to the implementation at hand [2]. SysMoBench therefore grades specifications in four phases of increasing demand: (1) syntax, (2) runtime model checking, (3) *transition validation* — replaying (*pre*, *action*, *post*) windows captured from real executions against the generated model — and (4) invariant verification.

SysMoBench was built for TLA^+ [3], and has since grown a pluggable backend interface with Alloy [5] and PAT/CSP# [6] backends. All three are formal languages, and all three are rare in LLM training corpora relative to mainstream programming languages. This asymmetry motivates the extension we present here.

The JS-SAM hypothesis. SAM [9, 10] is a software-engineering pattern whose semantics are explicitly derived from TLA^+ : *actions* compute proposals from inputs, a *model* accepts or rejects proposals as the sole locus of mutation, and *state* is a pure function of the model. A SAM specification is an ordinary executable JavaScript module. JavaScript is plausibly the single most abundant programming language in LLM training data. The hypothesis, then:

Can an LLM produce a correct specification of a real system more readily in a pattern hosted by a language it has seen constantly in training (JavaScript), than in formal languages it has rarely seen — while preserving the TLA^+ -style discipline that makes the artifact checkable?

Either answer is informative. If JS-SAM scores diverge upward, host-language familiarity dominates and formal syntax is a bottleneck; if scores match or fall, the hard part is modeling, not notation.

Contributions. This paper reports the first end-to-end results from the JS-SAM backend, all on the SysMoBench **spin** task (the Asterinas OS spinlock [12]):

1. **A non-formal-language backend for SysMoBench (§3):** the JS-SAM module contract, a sandboxed execution helper, and — a first for the benchmark — a *direct* Phase-3 replay path requiring no coding-agent orchestration, because SAM’s step relation *is* *model.present(action(data))*.
2. **A reproducible kernel trace-capture methodology (§4):** instrumenting the real Asterinas spinlock, driving a two-thread contention scenario in a single-CPU kernel test without

deadlock, and folding low-level kernel events into model-independent (*pre*, *action*, *post*) windows — producing the 28-window corpus used throughout.

3. **A four-model comparison** (§5) showing that Phases 1, 2, and 4 have no discriminating power on this task while Phase 3 spreads the models from 21.4% to 89.3% — and that general model capability does not predict modeling accuracy: the most capable model tested produced a specification that no-ops every lock release.
4. **A repair-loop experiment** (§6) demonstrating that self-correction from precise trace feedback is a sharper capability gradient than one-shot accuracy, separating the four models into three tiers.
5. **A JS-SAM-vs-TLA⁺ head-to-head** (§7): with identical models and task, 4/4 JS-SAM specifications reach end-to-end evaluability versus 1/4 for TLA⁺, with each TLA⁺ failure attributed to a specific structural cause.
6. **A controlled cross-language conformance study** (§8): a pre-registered, paired, four-arm design ($N=5$ generations per cell) that pins both languages to the same observable-state contract and replays the same trace corpus mechanically in both. After catching and controlling a prompt-prescriptiveness confound that would otherwise have produced a false headline, a plain-JavaScript control arm attributes the residual conformance gap to the SAM contract’s machinery — not to the JavaScript language, and not to formality — yielding a concrete design change for the backend.

We emphasize scope honestly: everything here concerns one task and a 28-window trace corpus, with single-generation results in Experiments 1–3 and $N=5$ replication only in Experiment 4. We present it as a case study whose value lies in the precision of the failure attribution and the reproducibility of the pipeline, not as a settled verdict on the hypothesis (§10).

2 Background

2.1 SysMoBench

SysMoBench [1] pairs eleven real systems (concurrent synchronization primitives and distributed protocols, from the Asterinas spinlock to Etcd Raft) with instrumentation harnesses and expert-written invariants. Given the system’s source, an LLM generates a specification, which is scored in four automated phases:

Phase 1 — Syntax. Does the specification parse and structurally validate? (For TLA⁺: the SANY analyzer.)

Phase 2 — Runtime model checking. Does bounded state-space exploration terminate without errors, deadlocks, or nondeterminism? (For TLA⁺: TLC [4].)

Phase 3 — Transition validation (conformance). Execution traces from the *real* system are cut into windows (*pre*, *action*, *post*); each window passes if the generated model, started at *pre* and applying *action*, produces *post*. The output is a per-action scorecard.

Phase 4 — Invariant verification. Expert invariants (e.g., mutual exclusion) are translated into the specification’s language and checked.

The benchmark’s central published finding is that frontier LLMs cluster near 100% on syntax but average roughly 46% on conformance and 41% on invariants [2]: models write valid TLA⁺ fluently yet describe the textbook protocol rather than the implementation. Phase 3 exists precisely to distinguish “writing TLA⁺” from “modeling this system.”

2.2 The SAM pattern

SAM (State–Action–Model) [9, 10] factors a program’s temporal behavior the way TLA⁺ factors a specification. An *action* is a pure function from inputs to a *proposal*; the *model* alone decides whether to accept a proposal and mutate itself (the analogue of a TLA⁺ next-state relation, with acceptors playing the role of guards); *state* is a pure function of the model. A step of the system is *model.present(action(data))*. The pattern is implemented by the **sam-pattern** library [11], which also provides a bounded explorer (**checker**) over the model’s reachable states.

Two properties make SAM suitable as a specification substrate. First, the semantic alignment with TLA⁺ means the benchmark’s phases map onto it without distortion: bounded exploration is Phase 2, replaying a trace window is literally one SAM step, and invariants are predicates over *getState()*. Second, the artifact is an executable JavaScript module: there is no separate tool chain between the specification and its own evaluation.

3 The JS-SAM backend

SysMoBench backends implement a **LanguageBackend** interface and register themselves; the CLI and the four evaluators resolve them by name with no further plumbing. The JS-SAM backend consists of a Python adapter and a small Node.js helper (**tools/js-sam/cli.mjs**) with four subcommands — **validate**, **check**, **transitions**, **invariants** — one per phase.

Module contract. The generation prompt instructs the model to emit a single CommonJS module:

```
module.exports = {
  instance,      // SAM instance (synchronous steps)
  init,          // () => void : reset to the initial state
  actions,       // { AcquireLock: (data)=>void, ReleaseLock: (data)=>void }
  getState,      // () => plain JSON-serializable snapshot
  setState,      // (snapshot) => void : force a state (Phase 3 replay)
  checkerIntents, // action descriptors + input domains (Phases 2 & 4)
};
```

actions must cover exactly the task’s target actions; **getState** must return a plain object (this is what trace states are diffed against); **setState** exists solely so Phase 3 can replay a window without searching for a path to its pre-state; and the module must be deterministic and I/O-free. **checkerIntents** declares each action’s finite input domain — for **spin**, every *{thread, callType}* combination — playing the role of a TLA⁺ constants block. Notably, there is *no separate configuration artifact*: the module is self-contained, a fact that becomes material in §7.

Sandboxing. Model-generated JavaScript is untrusted. Every helper invocation runs in a throwaway **node:20-slim** Docker container with no network, a read-only root filesystem, a non-root user, and CPU/memory/PID caps. The container is the trust boundary.

The direct Phase-3 path. For TLA⁺, SysMoBench’s transition validation drives TLC through a coding-agent workflow that constructs a checking module per window. JS-SAM needs none of that: the evaluator loads the module, and for each window executes `setState(pre); actions[name](data);` and diffs `getState()` against *post* under the benchmark’s projection rule (only keys present in the trace’s post-state must match, so model-internal bookkeeping is ignored). This “direct path” is the first non-agent Phase-3 implementation in the benchmark, and it follows from SAM’s semantics: a window *is* a SAM step.

Cross-platform engineering. Making the pipeline run on a Windows host (in addition to Linux/macOS) required fixing several host assumptions in the harness — POSIX-only `os.getuid()` calls, host-path container mounts invalid for `C:\` paths, and console encoding — plus threading the helper’s `stepsExplored` metric through a field previously hardcoded to zero for non-TLA⁺ backends. We also generalized the benchmark’s agent-based invariant translator (previously TLA⁺-only) into a shared, language-neutral component that JS-SAM can use, although all experiments below use the direct translator for parity across languages. These changes are upstreamed as pull requests.

4 Capturing ground truth: kernel traces for Phase 3

Phase 3 is the only phase requiring data external to the model, and SysMoBench does not ship trace corpora: traces are generated on demand by instrumenting the real system. Additionally, the maintainers’ own `spin` captures use three threads while the JS-SAM prompt models two, so the backend required its own corpus at the granularity it models. This section documents the pipeline; it is the experimental setup’s main methodological contribution and is fully reproducible from a single script.

4.1 Pipeline

1. **Pin a compatible revision.** The reference instrumentation patch applies cleanly at Asterinas v0.16.0 (matching the toolchain in the `asterinas/asterinas:0.16.0` image), not at `main`.
2. **Instrument.** Apply the benchmark’s reference patch, which adds a trace module to the kernel’s spinlock emitting one serial JSON event per lock operation, plus a new two-thread kernel test (below).
3. **Build and run.** Inside the container: build the `initramfs` and run the instrumented kernel test under QEMU (software emulation), collecting the event stream from the serial port.
4. **Parse.** Fold the event stream into (*pre*, *action*, *post*) NDJSON windows.

4.2 A two-thread contention scenario without deadlock

The reference spinlock kernel tests are single-actor: every operation is attributed to one thread, so contention — the behavior that makes a lock interesting — never appears in the trace. But a blocking `lock()` from a second actor in a single-CPU kernel test would spin forever. The scenario therefore expresses the second actor’s contention through non-blocking `try_lock` calls that fail without blocking:

```

actor0.lock()      -> TryAcquireBlocking(0), AcquireSuccess(0) // 0 holds
actor1.try_lock()  -> AcquireFail(1)                          // contention, no deadlock
actor0.release()   -> Release(0)
actor1.lock()      -> TryAcquireBlocking(1), AcquireSuccess(1) // 1 holds
actor0.try_lock()  -> AcquireFail(0)                          // contention
actor1.release()   -> Release(1)
actor0.try_lock()  -> AcquireSuccess(0)                       // try from free succeeds
actor0.release()   -> Release(0)

```

A second, longer scenario — more alternation of the holder, more failed and successful `try_lock` attempts, and same-thread reacquisition — extends the corpus from 8 windows to **28 windows** (17 `AcquireLock`, 11 `ReleaseLock`), the corpus used in all experiments below.

4.3 Model-independent state windows

The kernel emits low-level events (`TryAcquireBlocking`, `AcquireSuccess`, `AcquireFail`, `Release`). The parser folds these into windows keyed on the *objective* spinlock state — $\{lockHeld, lockHolder\}$, the real observable ownership — and relies on the projection rule to ignore any model’s internal bookkeeping (thread status, call type). A failed acquire under contention produces a window whose post-state ownership is unchanged; a successful acquire records the new holder together with whether it arrived via the blocking (`lock`) or non-blocking (`try`) path. Keying the corpus on observable state keeps the ground-truth oracle independent of any model under test.

5 Experiment 1: four models, four phases

5.1 Setup

Four Claude models spanning the capability range — Haiku 4.5, Sonnet 4.6, Opus 4.8, and Fable 5 (Anthropic’s most capable model at the time of writing) — each generated one JS-SAM specification for `spin` via a single prompt (`direct_call`), and the same artifact was scored in all four phases. Phase 2 and 4 exploration ran at the task’s default depth bound of 6; Phase 3 used the 28-window corpus; Phase 4 checked three expert invariants (`MutualExclusion`, `LockStatusConsistency`, `NoDeadlock`) translated via the direct API path.

5.2 Results

Table 1: Four-phase JS-SAM results on `spin` (28-window corpus). P3 per-action columns give windows passed / windows of that action.

Model	P1	P2 (states)	P3 overall	P3 Acquire	P3 Release	P4
Claude Opus 4.8	pass	pass (326,592)	89.3% (25/28)	14/17	11/11	3/3
Claude Fable 5	pass	pass (326,592)	50.0% (14/28)	14/17	0/11	3/3
Claude Sonnet 4.6	pass	pass (326,592)	50.0% (14/28)	14/17	0/11	3/3
Claude Haiku 4.5	pass	pass (326,592)	21.4% (6/28)	6/17	0/11	3/3

Two results stand out.

Only the reality check discriminates. Every model passes Phase 1, Phase 2 (all exploring the identical 326,592 states, confirming structurally equivalent state spaces), and Phase 4. On this task those three phases have *zero* discriminating power. Phase 3 spreads the models across a 4× range, and — restricted to comparisons it can make — orders Opus above the mid tier above Haiku. This independently reproduces, in a non-formal language, the central SysMoBench finding for TLA⁺: internal consistency is cheap; conformance with the real system is where specifications fail [2].

General capability does not predict modeling accuracy. The most striking row is Fable 5, the most capable model tested, scoring 50% — below Opus 4.8 and tied with mid-tier Sonnet. The cause is specific and systematic, not noise: Fable’s specification handles acquisition competently (14/17) but *no-ops every single release* (0/11). After any **ReleaseLock**, its model leaves the lock held:

ReleaseLock	expected	{lockHeld: false, lockHolder: null}
	got	{lockHeld: true, lockHolder: 0}

The release logic *reads* correctly in isolation — it guards on the holder, then clears the state — but on replay the release never takes effect: a categorical, whole-action modeling defect. Crucially, this specification passed syntax, explored 326,592 states without violation, and satisfied all three invariants, including mutual exclusion. (A lock that is never released trivially preserves mutual exclusion.) Only replay against the real kernel’s transitions exposed it.

The recurring failure among the passing acquisitions is also precise: all three 14/17 models fail exactly the three **try_lock**-from-free windows — the non-blocking acquire that should succeed on a free lock. Blocking acquisition (abundant in training data and in textbook treatments) is modeled correctly by every model except Haiku; the less common non-blocking path is where they slip. This echoes, at micro scale, SysMoBench’s “textbook modeling” diagnosis: models reproduce the familiar template and miss the implementation-specific path.

6 Experiment 2: repair from trace feedback

6.1 Setup

If a model is shown *exactly* which transitions its specification got wrong — the pre-state, the action and its data, the real system’s post-state, and what the model produced instead — can it fix the specification? For each model we ran one repair round: score the original specification on Phase 3; build a repair prompt containing the full original module plus every failing window; ask the model to rewrite the module; confirm the result still passes Phase 1; re-score Phase 3 on the same 28-window corpus.

6.2 Results

Self-correction is a clean three-tier gradient. Opus and Fable repair to a perfect 28/28 in a single round. Sonnet repairs *partially*: it fixes the **try_lock** acquisition defect completely but cannot fix its release path even when handed the exact failing transitions — it uses one class of feedback and not the other. Haiku does not improve at all. The repair axis separates the four models into three distinct tiers (full / partial / none), more cleanly than the baseline scores, which conflate Fable and Sonnet at 50%.

Table 2: One-round repair from Phase-3 feedback (same 28-window corpus).

Model	Baseline P3	Repaired P3	Acquire (base→rep)	Release (base→rep)
Claude Opus 4.8	89.3% (25/28)	100% (28/28)	82% → 100%	100% → 100%
Claude Fable 5	50.0% (14/28)	100% (28/28)	82% → 100%	0% → 100%
Claude Sonnet 4.6	50.0% (14/28)	60.7% (17/28)	82% → 100%	0% → 0%
Claude Haiku 4.5	21.4% (6/28)	21.4% (6/28)	35% → 35%	0% → 0%

The Fable reversal. Fable’s baseline looked no better than Sonnet’s, and its failure mode — a whole action silently broken — looked worse. Yet under repair, Fable fully recovers while Sonnet, given identical feedback for the identical symptom (Release 0/11), cannot. This reframes the baseline: Fable’s release defect was a *recoverable slip*, not a capability ceiling, whereas Sonnet’s is the latter. One-shot specification accuracy and repair-from-feedback are different axes of capability, and a benchmark that measures only the former will misrank models for any workflow that includes a correction loop — which realistic agentic formal-modeling workflows do [2].

What this does and does not measure. The repair prompt contains the failing windows themselves, so the experiment measures “can the model exploit precise, correct feedback,” not blind improvement. That is the intended question — it is exactly the signal available inside an agentic repair loop — but it is not a from-scratch re-test, and we ran a single round; whether Sonnet or Haiku converge with more rounds is untested.

7 Experiment 3: JS-SAM versus TLA⁺

7.1 Setup

The head-to-head the backend exists to inform: the same four models, the same `spin` task, one `direct_call` generation per model per language, scored on Phase 1 (syntax: SANY vs. module validation), Phase 2 (model checking: TLC vs. bounded exploration), and Phase 4 (invariant verification, direct translator for both languages). Phase 3 is deliberately *not* compared: JS-SAM’s Phase 3 is a self-contained direct replay while TLA⁺’s is a heavyweight coding-agent path — different mechanisms, not a fair one-to-one. Phase 4 is compared as pass/fail because the two languages ship different invariant-template counts for `spin` (seven for TLA⁺, three for JS-SAM).

7.2 Results

Table 3: JS-SAM vs. TLA⁺ on `spin`: models producing a passing artifact per phase.

	P1 syntax	P2 model-checkable	P4 invariants
JS-SAM	4/4	4/4	4/4
TLA ⁺	4/4	1/4	1/4

Syntax is not where the languages differ. Every model writes syntactically valid specifications in *both* languages — consistent with SysMoBench’s finding that frontier models have

Table 4: Per-model TLA⁺ detail.

Model	P1	P2	P4	P2 failure cause
Claude Opus 4.8	pass	fail	fail	unbounded CHOOSE (TLC rejects)
Claude Fable 5	pass	pass (574 states)	pass (7 inv.)	—
Claude Sonnet 4.6	pass	fail	fail	unbounded CHOOSE (TLC rejects)
Claude Haiku 4.5	pass	fail	fail	config generation fell back; no usable .cfg

essentially saturated TLA⁺ syntax [2]. The divergence is at Phase 2: whether the specification is actually *checkable*. All four JS-SAM modules run; one of four TLA⁺ specifications does. Each TLA⁺ failure is attributable to a specific structural cause.

Cause 1: a TLA⁺ semantic pitfall JavaScript does not have. Opus and Sonnet both modeled “no lock owner” as `NULL == CHOOSE v : v \notin Threads`. SANY accepts this — syntax passes — but TLC cannot evaluate an unbounded CHOOSE, so model checking fails. The idiom is natural, looks correct, and is a well-known TLC trap. JavaScript has a native `null`, so the corresponding JS-SAM specifications express the same concept with no trap available to fall into. This structural hazard caught two of four models, *including the model with the best JS-SAM conformance score* (Opus). It is a clean instance of the hypothesis’s mechanism: the failure is not “the model cannot model a spinlock” (its JavaScript model of the same system scored 89.3% against real traces) but “the formal language exposes a semantic surface on which competent models slip.”

Cause 2: a configuration surface JS-SAM does not have. TLA⁺ model checking requires a separate .cfg artifact (constants, init/next, invariants) that the harness must generate to match the specification; for Haiku, that generation fell back and produced no usable configuration (zero states explored). A JS-SAM module is self-contained and executable — there is no second artifact to generate, so this entire failure mode is structurally absent. We flag the attribution honestly: the CHOOSE failures belong to the models; the Haiku failure belongs to the harness. But the asymmetry itself is structural — one language’s evaluability depends on a fragile secondary artifact and the other’s does not.

Net. On this task, models reach an end-to-end evaluable specification far more readily in JS-SAM (4/4 through Phase 4) than in TLA⁺ (1/4). This is direct — though single-task — evidence for the hypothesis in its practical form: *host-language familiarity plus a self-contained executable artifact lowers the barrier to a usable specification*. It does not, by itself, show that JS-SAM specifications are more *faithful*; it shows they are far more often checkable at all, which is the precondition for faithfulness to be measured. Experiment 4 (§8) measures faithfulness directly, under a design built to make the two languages comparable — and resolves it in an unexpected place.

8 Experiment 4: cross-language conformance under a controlled contract

Experiment 3 could not compare Phase 3 across languages because the two backends implement it by incomparable mechanisms: JS-SAM runs a mechanical replay, while TLA⁺ runs a coding-agent path — necessary because a free-form TLA⁺ specification invents its own variable names, so mapping trace states onto spec states requires interpretation. A naive comparison would measure the replay machinery, not the specifications. This experiment removes that asymmetry with a pre-registered, paired design; the full methodology note, written to be socialized with the benchmark maintainers, is included in the artifact repository.

8.1 Design

Equalize the observable-state contract. The degree of freedom that forces TLA⁺ onto the agent path — free choice of variables — is one the JS-SAM prompt already removes by pinning `getState()`’s shape. We remove it symmetrically: a constrained TLA⁺ prompt mandates `VARIABLES lockHeld, lockHolder` (the trace schema), action operators `AcquireLock(thread, callType) / ReleaseLock(thread)`, and a TLC-safe sentinel (`NONE == "none"`, not `CHOOSE`). Both languages then answer the identical question — *express this system over this observable state* — and trace states map onto spec states with no interpretation.

A direct TLC replay path (no agent). With the contract fixed, TLA⁺ transition validation becomes mechanical, mirroring JS-SAM’s `setState` → action → diff: for each window, a synthesized module pins the pre-state in its initial predicate, takes exactly one step of the window’s action (then stutters), and asserts via an invariant that the post-state is unreachable; a TLC-reported violation means the window *passes*. The replay was validated in both directions before use: a contract-following specification passes 28/28 (positive control), and the same specification with `ReleaseLock` mutated to a no-op fails exactly the 11 release windows, 17/17 acquires still passing (negative control). The replay discriminates; it does not trivially pass everything.

Four arms, because the prompt and the contract are both confounds. An initial two-arm run (deployed JS-SAM prompt vs. constrained TLA⁺) produced what looked like a decisive TLA⁺ reversal. But the constrained TLA⁺ prompt *states the exact single-step observable semantics* (free → acquire; held → unchanged; holder → release), which the deployed JS-SAM prompt does not: a difference in prompt content, not in language. And even with prompts equalized, the two substrates still differ in a second way — the SAM contract obliges an executable module with proposal/acceptor wiring and auxiliary state, while the TLA⁺ specification is a bare two-variable relation. We therefore ran four arms: **JS(deployed)** — the shipped JS-SAM prompt; **JS(constrained)** — the deployed prompt plus the *identical* semantics block used by the TLA⁺ arm; **plain-JS** — the same constrained semantics, but the specification is a bare `next(state, action, data)` transition function over only `{lockHeld, lockHolder}`: JavaScript, yet structurally isomorphic to the TLA⁺ relation, with no SAM library and no auxiliary state; and **TLA(constrained)**. JS(constrained) vs. TLA(constrained) varies language and contract together; plain-JS vs. TLA(constrained) varies *only* the language; JS(constrained) vs. plain-JS varies *only* the contract.

Replication, pairing, and checkability. $N=5$ generations per model per arm (80 in total), all scored per-window on the same 28-window kernel corpus. The design is naturally paired — for each model, every window has an outcome in every arm — so we report per-model McNemar tests (continuity-corrected, $\chi^2 > 3.84 \Rightarrow p < 0.05$) rather than comparing noisy means. The pre-registered checkability policy (conditional vs. unconditional scoring) proved moot: *zero* windows were unscorable in any constrained or plain-JS generation.

8.2 Results

Table 5: Cross-language Phase-3 conformance (mean pass rate over $N=5$ generations per cell; 28-window corpus). Conditional and unconditional rates coincide — no unscorable windows.

Model	JS (deployed)	JS (constrained)	plain-JS	TLA ⁺ (constrained)
Claude Opus 4.8	81.4%	89.3%	100%	100%
Claude Fable 5	57.9%	95.7%	100%	100%
Claude Sonnet 4.6	50.0%	100%	100%	100%
Claude Haiku 4.5	28.6%	40.7%	100%	100%

Table 6: Paired McNemar over windows, aggregated across generations (b = JS-only pass, c = TLA⁺-only pass; * = $p < 0.05$). The plain-JS arm is a perfect per-window tie with TLA⁺ for every model ($b=0$, $c=0$, $\chi^2=0.0$) and is omitted from the table.

Model	JS(deployed) vs. TLA	JS(constrained) vs. TLA
Opus 4.8	$b=0$, $c=26$, $\chi^2=24.0^*$	$b=0$, $c=15$, $\chi^2=13.1^*$
Fable 5	$b=0$, $c=59$, $\chi^2=57.0^*$	$b=0$, $c=6$, $\chi^2=4.2^*$
Sonnet 4.6	$b=0$, $c=70$, $\chi^2=68.0^*$	$b=0$, $c=0$, $\chi^2=0.0$
Haiku 4.5	$b=0$, $c=100$, $\chi^2=98.0^*$	$b=0$, $c=83$, $\chi^2=81.0^*$

8.3 Findings

Most of the apparent TLA⁺ advantage was prompt prescriptiveness, not language. Adding the same semantics block to the JavaScript prompt — changing nothing else — moved Fable from 57.9% to 95.7% and Sonnet from 50.0% to a perfect 100%. Had we reported the two-arm run, the headline would have been a decisive language reversal; most of that effect was the prompt. We regard catching this as a result in its own right: *prompt prescriptiveness is a first-class experimental variable in specification-generation benchmarks*, large enough to masquerade as a language effect.

A real, one-directional residual survives the prompt control. With identical semantics in both prompts, TLA⁺ reaches 100% conformance for *every model and every generation*, while constrained JS-SAM does so only for Sonnet. The residual is statistically significant for Opus, Fable, and Haiku, vanishes for Sonnet, and is largest for the weakest model (Haiku: 40.7% vs. 100%). Moreover $b=0$ in every comparison: no JS-SAM specification ever passed a window that its TLA⁺ counterpart failed. (When one arm is at 100% this is partly structural — see §10 — but the direction is uniform in the deployed arm as well.)

The plain-JS arm locates the residual: it is the contract, not the language. The fourth arm ties TLA^+ at 100% for every model and every generation — $b=0$, $c=0$, $\chi^2=0.0$ for all four models: perfect per-window ties across all paired generation-sets. Haiku is the cleanest single datum: the weakest model, which reaches only 40.7% through SAM’s machinery, expresses the identical semantics perfectly as a bare transition function — in the same language. Three attributions fall out. *It is not the language*: same JavaScript, different shape, 100%. *It is not formality*: TLA^+ and plain-JS share the same shape — a minimal declarative single-step transition over exactly the observable state — and both sit at 100%; what the three-arm data suggested was “formal directness” is really *minimal declarative shape*. *It is the SAM contract’s defect surface*: proposal/acceptor wiring, intent firing, and the mandatory auxiliary state (`threadStatus`, `callType`) — precisely where Experiment 1’s `try_lock`-from-free and release bugs lived. Every moving part the contract obliges is transcription surface, and the tax falls hardest on the weakest model (Haiku pays 59 points; Sonnet pays nothing).

Verdict against the pre-registered outcomes. Not the strong “JS-SAM wins” (the direction is reversed), and not a tie at the backend level — but at the *language* level, a tie is exactly what the controlled comparison shows: JavaScript expresses the transition relation as reliably as TLA^+ once the specification shape is matched. The pre-registered outcome list did not anticipate this resolution; we note for transparency that the plain-JS arm was designed after the three-arm results were known (though scored blind, by the same mechanical replay, on the same corpus).

Design implication for JS-SAM. The Phase-3 fidelity gap is the cost of the SAM specification contract, not a language limit — and the two can be decoupled. The backend can offer a lean specification shape (`next(state, action, data)`, the JavaScript analogue of the TLA^+ relation), or score conformance against a plain transition core over only the observable variables, dropping the mandatory auxiliary state from the replay contract. SAM’s ceremony is not gratuitous — the instance, intents, and value domains are exactly what feed the Phase-2/4 bounded explorer — but it should be paid where it buys something: explorable structure for model checking, not conformance replay. A prototype confirms the decoupling is practical: a reference lean specification plus a ~ 40 -line generic breadth-first explorer over `{init, next}` runs the entire pipeline in the same locked-down sandbox — Phase 2 passes, all three Phase-4 invariants hold, and Phase 3 scores 28/28 — so the SAM machinery is not *required* by any phase. (The caveat: `spin`’s observable state is tiny — three reachable states — so the lean explorer’s adequacy must be re-validated on richer tasks before generalizing.)

Checkability, revisited. Zero unscoreable generations in the constrained TLA^+ arm confirms Experiment 3’s attribution: the earlier “1/4 model-checkable” was entirely the unbounded-CHOOSE trap plus the configuration surface, both of which the pinned contract and direct replay remove. The checkability gap is real but shallow — a matter of guardrails, not capability.

A free variance check on Experiment 1. The JS(deployed) arm replicates Experiment 1’s setting at $N=5$. The single-generation scores (89.3, 50.0, 50.0, 21.4) sit near the five-run means (81.4, 57.9, 50.0, 28.6), and the model ordering is preserved (with Fable’s mean now slightly above Sonnet’s). Experiment 1’s qualitative conclusions survive replication; its exact figures move by up to ~ 8 points, which is the error bar readers should apply to any single-generation number in this paper.

9 Discussion

“Looks right” is not “is right,” in any language. The four experiments repeatedly produced specifications that passed every internal-consistency check while being wrong about the real system — most vividly Fable’s never-releasing lock, which preserved mutual exclusion *because* it was broken. Phase 3’s value is not incremental rigor; it is the only phase whose ground truth is external to the model. This replicates SysMoBench’s core claim [1, 2] in a new language substrate, which strengthens it: the phenomenon is about LLM modeling, not about TLA⁺.

The familiar path is modeled; the unfamiliar path is not. For the three stronger models, every Experiment-1 failure lands on a less-common code path: the non-blocking `try_lock`-from-free acquisition (all three) and release bookkeeping (two of the three). Blocking acquisition — the version of a spinlock every textbook presents — was modeled correctly by all but the smallest model. This is the “textbook template” failure mode of [2] reproduced at the granularity of individual actions within a 100-line system.

Repair belongs in the evaluation loop. The repair experiment suggests that single-shot leaderboard scores understate some models (Fable) and overstate the ceiling of others (Sonnet, Haiku). Since practical formal-modeling deployments will be agentic loops with exactly this kind of counterexample feedback, benchmarks arguably should report both axes. The repair driver we contribute makes the second axis cheap to measure for any JS-SAM task.

The hypothesis, after Experiment 4: a split verdict, sharply located. The practical form of the hypothesis survives: host-language familiarity plus a self-contained executable artifact gets models to an evaluable specification far more often (4/4 vs. 1/4), and evaluability is the precondition for everything else. The strong form — that familiarity yields more *faithful* models — resolves to a tie once like is compared with like: JavaScript in the shape of the TLA⁺ relation matches TLA⁺ at 100% for every model. What the three-arm data made look like a language effect was the specification contract: every layer of machinery between the semantics and the artifact — proposals, acceptors, auxiliary state — is surface on which transcription slips, and the tax falls hardest on the weakest models. Neither training-data abundance nor formality is the operative variable on this task; *contract minimality* is. And the largest single effect in the study is still the prompt (Sonnet 50%→100%): prompt, then contract, then language is the order of what mattered here.

10 Limitations and threats to validity

We state these plainly; the study is a case study and should be read as one.

- **One task.** Everything here concerns a single system — a spinlock, the *easiest* class of SysMoBench task. Distributed protocols with richer state (Raft variants, ring buffers, lock services) may reorder every result. The spinlock’s simplicity is also why Phases 1/2/4 saturate; on harder tasks they discriminate too [2].
- **Experiments 1–3 are single-generation.** Each cell in Tables 1–4 is one sample. Experiment 4’s $N=5$ deployed-JS arm retroactively bounds the damage — ordering preserved,

individual scores moving by up to ~ 8 points — but the repair-loop and checkability experiments have not been replicated and should be read with that error bar.

- **28 trace windows from two deterministic scenarios.** Contention is expressed via `try_lock` on a single CPU; true SMP interleavings and a blocking waiter’s wakeup are not represented. The corpus is faithful to real kernel executions but does not exhaust the lock’s behavior.
- **The TLA⁺ comparison mixes causes.** The 1/4-vs-4/4 result bundles a genuine model-attributable pitfall (unbounded `CHOOSE`, 2/4) with a harness shortfall (config generation, 1/4). We report the attribution per model (Table 4); the headline number should not be read as purely a language effect. No repair round was run on the TLA⁺ side — a repaired Opus might fix its `CHOOSE` readily, and symmetry demands that experiment before strong claims.
- **Experiment 4 measures transcription, not derivation.** With the semantics block present, the constrained and plain arms measure how reliably a model *transcribes* near-spec-level semantics into a language, not whether it can *derive* the semantics from source code. The deployed-JS arm is the closer proxy for derivation — and there the gap is much larger — but there is no deployed-TLA⁺ arm to pair it with (free-form TLA⁺ was mostly uncheckable), so the derivation-level comparison remains open.
- **The plain-JS arm resolves the substrate asymmetry, but was added post hoc and answers a narrower question.** The three-arm design left “executable machinery admits bugs a two-variable relation cannot” as an open alternative; the fourth arm was designed to test exactly that, after the three-arm results were known (its data were then collected by the same blind mechanical replay). Its conclusion is also bounded: a bare `next()` function carries no SAM instance, intents, or value domains, so it feeds Phase 3 only — the lean shape’s cost at Phases 2 and 4 is unmeasured, and the “contract tax” finding applies to conformance, not to the whole benchmark.
- **$b=0$ is partly structural.** When the TLA⁺ arm passes every window it fails none, so a JS-only pass is impossible by construction; the McNemar then reduces to whether the JS-SAM failure count is significant. We report it for completeness, not as an independent directional finding.
- **The direct TLC replay is the controlled condition, not the deployed one.** The benchmark ships an agent-mediated TLA⁺ Phase 3; pinning spec variables to the trace schema trades realism for comparability. Whether a “pinned-schema” Phase-3 mode should exist as a first-class controlled condition alongside the agent path is a methodological question we are putting to the maintainers.
- **Phase 4 differs in template count** (seven TLA⁺ invariants vs. three JS-SAM), so it is compared pass/fail only, and the JS-SAM invariant set is the weaker filter.
- **Model family.** All four models are from one vendor. The benchmark’s configuration already includes other vendors’ models; the comparison should be broadened before generalizing.

- **Author positionality.** The first author is the author of the SAM pattern. The evaluation pipeline is automated and the corpus is model-independent, which limits (but does not eliminate) the room for favorable design choices; all code, prompts, traces, and drivers are public for scrutiny. We note that the study’s sharpest negative finding — the contract-tax attribution — cuts *against* the SAM contract as deployed, which we take as evidence the pipeline is not tilted toward it.

11 Related work

Benchmarking LLM formal modeling. SysMoBench [1, 2] is the direct foundation of this work: it contributes the four-phase methodology, the eleven-system task suite, and the finding that conformance — not syntax — is where LLM specifications fail. Our contribution is orthogonal: holding the benchmark fixed and varying the specification *language*, including the first non-formal one. The Specula agent [13] shows that agentic workflows can saturate current SysMoBench tasks in TLA^+ ; our repair experiment probes the smallest unit of such a workflow (one counterexample round) across models.

Trace conformance for specifications. Validating specifications against implementation traces has industrial precedent: MongoDB’s eXtreme modeling checked TLA^+ models against driver traces [8], and trace validation is the standard answer to “does the spec match the code” [1]. SysMoBench’s transition-window formulation, which we adopt unchanged, makes conformance per-action and per-window rather than whole-trace.

Formal methods practice. TLA^+ [3] with TLC [4], Alloy [5], and CSP-based PAT [6] are the benchmark’s formal backends; industrial deployments (e.g., [7]) motivate the whole enterprise. SAM [9, 10] occupies an unusual position: an engineering pattern with TLA^+ -derived semantics, here repurposed as a specification language precisely because it sits at the intersection of “formally disciplined” and “abundant in training data.”

12 Conclusion and future work

We extended SysMoBench with JS-SAM, its first non-formal specification backend, built the kernel trace-capture pipeline needed to give it a real conformance phase, and ran four experiments on the Asterinas spinlock task. The results make four points with unusual precision for a single case study: transition validation against real traces is the only phase that discriminated among four frontier models (21.4%–89.3%, with all other phases unanimous); one round of trace-feedback repair separates models into full/partial/no self-correction tiers that one-shot scores conceal; the same models that reach an evaluable specification in JavaScript 4/4 times manage it in TLA^+ 1/4 times, for two identifiable structural reasons; and under a pre-registered, paired, four-arm controlled comparison, the faithfulness question resolves in an unexpected place — most of an apparent TLA^+ conformance advantage was prompt prescriptiveness, and the residual that survives is attributable to the SAM contract’s machinery, not to JavaScript and not to formality: plain JavaScript in the shape of the TLA^+ relation ties TLA^+ at 100% for every model.

The hypothesis that motivated the backend is neither confirmed nor dead; it is sharpened twice over. Familiarity buys *checkability*; contract minimality buys *transcription fidelity*; and the prompt buys more than either. The experiments that follow are concrete: (1) promoting

the lean specification shape from prototype to backend: the reference implementation (a generic `{init, next}` explorer that already passes all phases on `spin`) supports three integration paths — a first-class lean backend, a hybrid where the model writes only the pure core and the harness generates the SAM scaffolding, or the documented status quo — and should next be exercised with model-generated specifications on richer tasks; (2) a derivation-level comparison — both languages, *no* semantics block, on a task rich enough that the semantics cannot be guessed from the action names (ring buffer or lock service), which remains the decisive open test of the familiarity claim; (3) a prompt-prescriptiveness dose–response study, treating the semantics block as the manipulated variable rather than a nuisance; (4) an as-deployed agent-path arm beside the controlled replay, for ecological validity; (5) a TLA⁺ repair round for symmetry with Experiment 2; and (6) replication beyond one model family. We are contributing the backend, harnesses, drivers, corpora, and the controlled-comparison methodology upstream so these can be run — and challenged — by the community.

Acknowledgments

We thank the SysMoBench / Specula maintainers for the benchmark, the integration guidance that shaped the JS-SAM backend’s scope, and the reference instrumentation for the Asterinas spinlock.

Reproducibility

All artifacts are public: the JS-SAM backend (`tla_eval/languages/js_sam.py`, `tools/js-sam/`), the trace harness (`scripts/harness/spin/`), the captured corpus (`data/sys_traces/spin/`), the repair driver (`scripts/repair_phase3.py`), the Experiment-4 apparatus — the constrained and plain-JS prompts, the direct TLC replay (`scripts/tla_direct_tv.py`), the paired four-arm driver with raw per-window data (`scripts/tla_phase3_study.py`, `output/tla_phase3_study.json`), the pre-registered design and results (`docs/js_sam_tla_phase3_*.md`), the lean-contract prototype (`tools/plain-js/`, `scripts/plain_js_tv.py`, `docs/js_sam_lean_contract.md`), and the methodology note (`paper/methodology.md`) — and the experiment write-ups (`docs/js_sam_*.md`) in the SysMoBench fork at <https://github.com/jdubray/SysMoBench-1>, staged as stacked pull requests against <https://github.com/specula-org/SysMoBench>. Requirements: Docker, Node ≥ 20 , Python 3, and (for the TLA⁺ arm) Java with `tla2tools`. A single script reproduces the kernel capture: `scripts/harness/spin/run.sh`.

References

- [1] Q. Cheng, R. Tang, E. Ma, F. Hackett, P. He, Y. Su, I. Beschastnikh, Y. Huang, X. Ma, and T. Xu. SysMoBench: Evaluating AI on formally modeling real-world systems. arXiv:2509.23130, 2025. Leaderboard: <https://sysmobench.com>.
- [2] Q. Cheng, R. Tang, E. Ma, F. Hackett, P. He, Y. Su, I. Beschastnikh, Y. Huang, X. Ma, and T. Xu. Can LLMs model real-world systems in TLA+? ACM SIGOPS Blog, May 2026. <https://www.sigops.org/2026/can-llms-model-real-world-systems-in-tla/>.
- [3] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

- [4] Y. Yu, P. Manolios, and L. Lamport. Model checking TLA+ specifications. In *CHARME*, 1999.
- [5] D. Jackson. Alloy: A lightweight object modelling notation. *ACM TOSEM*, 11(2), 2002.
- [6] J. Sun, Y. Liu, J. S. Dong, and J. Pang. PAT: Towards flexible verification under fairness. In *CAV*, 2009.
- [7] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. How Amazon Web Services uses formal methods. *Communications of the ACM*, 58(4), 2015.
- [8] A. J. Davis, M. Hirschhorn, and J. Schvimer. eXtreme modelling in practice. *PVLDB*, 13(9), 2020.
- [9] J.-J. Dubray. The SAM pattern. <https://sam.js.org>.
- [10] J.-J. Dubray. Why I no longer use MVC frameworks. InfoQ, February 2016. <https://www.infoq.com/articles/no-more-mvc-frameworks/>.
- [11] J.-J. Dubray. sam-pattern: a library implementing the SAM pattern. <https://www.npmjs.com/package/@cognitive-fab/sam-pattern>.
- [12] Asterinas project. Asterinas: a Linux ABI-compatible, Rust-based framekernel OS. <https://github.com/asterinas/asterinas>.
- [13] Specula project. Specula: an agent specialized in TLA+ formal modeling. <https://github.com/specula-org/Specula>.